PROGRAMARE ORIENTATĂ PE OBIECTE

şl.ing. Carmen ODUBĂŞTEANU

CUPRINS

- Introducere în limbajul Java
- Programare orientată pe obiecte – concepte generale
- Programare orientată pe obiecte în limbajul Java
  - Obiecte, Clase, Constructori, Modificatori de acces
  - Polimorfism, **Agregare/ Moştenire, Upcasting/ Downcasting**
  - **Legare statică/legare dinamică**
- Excepţii
- Fluxuri (intrări-ieşiri)
- Clase abstracte şi Interfeţe
- Clase incluse
- Colecţii
- Genericitate
- Design Patterns
- Aplicatii concepte POO: Interfete grafice-Swing

BIBLIOGRAFIE

- curs.upb.ro – curs
- https://ocw.cs.pub.ro/courses/poo – laborator + curs
- Curs practic de Java, Cristian Frăsinaru
- Programare orientată pe obiecte in Java, Florian Moraru, Carmen Odubăşteanu
- Java de la 0 la Expert, Ştefan Tanasa, s.a.
- Java - o perspectiva pragmatică, Irina Athanasiu, s.a.
- Java Tutorial,

  https://docs.oracle.com/javase/tutorial/
- Thinking in Java, Bruce Eckel, www.bruceckel.com

Punctaj

1. Laborator
   - Prezenta (cu activitate) – 1.2p
   - Activitate – 1.3p
2. Tema – 2p

Total parcurs: 4.5p

Minim 2p pe parcurs pentru intrare in examen

3. Examen – 5.5p

Introducere în Java

- Tehnologia Java
- Primul program
- Structura lexicală
- Tipuri de date
- Variabile
- Instrucţiuni
- Vectori
- Şiruri de caractere
- Argumente in linia de comandă
- *Ce este un pachet ?*
- *Pachetele standard (J2SDK)*
- *Importul unei clase sau interfeţe*
- *Importul la cerere*
- *Importul static*
- *Crearea unui pachet*
- *Organizarea fişierelor*

Limbajul de programare Java

- Simplitate
- Uşurinţă în crearea de aplicaţii complexe
- Robusteţe – nu există pointeri, administrarea automată a memoriei, GC
- Complet orientat pe obiecte
- Securitate
- Neutralitate arhitecturală
- Portabililtate
- Compilat şi interpretat
- Performanţă
- Modelat după C şi C++

Platforme de lucru Java

- J2SE (Standard Edition)

Aplicaţii independente, appleturi, Java Web Start

- J2ME (Micro Edition)

Programarea dispozitivelor mobile

- J2EE (Enterprise Edition)
  - Aplicaţii complexe, pe mai multe nivele pentru sisteme eterogene, cu baze de date distribuite, etc.
  - Aplicaţii şi servicii Web: servleturi, pagini JSP, etc.
- Distribuţiile Java sunt oferite gratuit
- www.oracle.com/technetwork/java/javase/downloads/

Primul program

1. Scriererea codului sursă

```
class FirstApp
{
  public static void main ( String args[])
  {
      System.out.println("Hello world!");
   }
}
```

Definire clasă – class *numeclasa*

- Funcţia principală a unei aplicaţii Java propriu-zise - *public static void main(String args[])*
- Afişare - *System.out.println*

Primul program

2. Salvarea fişierelor sursă
   C:\intro\FirstApp.java

3. Compilarea aplicaţiei
   javac FirstApp.java
   va rezulta: FirstApp.class

4. Rularea aplicaţiei
   java FirstApp

- Se poate folosi un IDE (mediu integrat de dezvoltare) pentru a realiza toţi paşii anteriori (JCreator, Eclipse, NetBeans, etc.)

Structura lexicală

Setul de caractere: Unicode

- înlocuieşte ASCII
- un caracter se reprezintă pe 2 octeţi
- conţine 65536 de semne
- compatibil ASCII : primele 256 caractere sunt cele din ASCII
- structurat în blocuri:

  Basic, Latin, Greek, Arabic, Gothic, Currency, Mathematical, Arrows, Musical, etc.
- referire prin cod hexa:

  \uxxxx

  \u03B1 -\u03C9: α - ω
- http://www.unicode.org

Cuvinte cheie

- Cuvintele rezervate sunt, cu câteva excepţii, cele din C++.
  - abstract, double, int, strictfp
  - boolean, else, interface, super
  - break, extends, long, switch
  - byte, final, native, synchronized
  - case, finally, new, this
  - catch, float, package, throw
  - char, for, private, throws
  - class, goto*, protected, transient
  - const*, if, public, try
  - continue, implements, return, void
  - default, import, short, volatile
  - do, instanceof, static, while

Incepând cu 1.5: enum.

Identificatori

- Sunt secvenţe nelimitate de litere şi cifre Unicode plus simbolul “_”, ce încep cu o literă sau “_”.
- Identificatorii nu au voie să fie identici cu cuvintele rezervate.

Exemple: a1, FirstApp, factorial, etc.

Convenţie:

- Nume de clasa: prima literă mare (Complex)
- Nume de metodă: prima literă mică (aduna, adunaComplex)
- Nume variabilă: prima literă mică (var1)
- Nume constantă: prima literă mare sau tot numele cu litere mari (Pi, PI)

Obs: *dacă identificatorul este format din mai mulţi atomi lexicali, atunci primele litere ale celorlalţi atomi se scriu cu majuscule.*

Constante

- Intregi (10, 16 -0x, 8-0)
  - normali - se reprezintă pe 4 octeţi (32 biţi)
  - lungi - se reprezintă pe 8 octeţi (64 biţi) şi se termină cu caracterul L (sau l).

- Flotanţi: 1.0, 2e2, 3f, 4D
  - să aibă cel puţin o zecimală după virgulă
  - să fie în notaţie exponenţială
  - să aibă sufixul F sau f pentru valorile normale - reprezentate pe 32 biţi, respectiv D sau d pentru valorile duble - reprezentate pe 64 biţi.

- Logici: true, false

Constante

- Caracter: ’J’, ’a’, ’v’, ’a’, ’\n’
- Caracter sau secvenţe escape (permit specificarea caracterelor care nu au reprezentare grafică şi reprezentarea unor caractere speciale precum backslash, apostrof, etc.)
- Secvenţele escape predefinite în Java sunt:
  - ’\b’ : Backspace (BS)
  - ’\t’ : Tab orizontal (HT)
  - ’\n’ : Linie nouă (LF)
  - ’\f’ : Pagină nouă (FF)
  - ’\r’ : Inceput de rând (CR)
  - ’\"’ : Ghilimele
  - ’\’’ : Apostrof
  - ’\\’ : Backslash

Constante

- Şiruri de caractere: ”Text”
  - format din zero sau mai multe caractere între ghilimele. Caracterele care formează şirul pot fi caractere grafice sau secvenţe escape.

- Separatori: indică sfârşitul unei unităţi lexicale şi începutul alteia.

  ( ) [ ] ; , .

Observaţie: instrucţiunile sunt separate prin ;

Operatori

- atribuirea:
  =
- matematici:
  +, -, *, /, %, ++, --
  lval op= rval: x += 2 n -= 3
  x++, ++x, n--, --n
- logici:
  &&(and), ||(or), !(not)
- relaţionali: <, <=, >, >=, ==, !=
- pe biţi:
  &(and), |(or), ^ (xor),~ (not)
- de translaţie: <<, >>, >>> (shift la dreapta fără semn)
- if-else:
  expresie-logica ? val-true: val-false

Operatori

- operatorul , (virgulă) folosit pentru evaluarea secvenţială a operaţiilor:

```
int x=0, y=1, z=2;
```

- operatorul + pentru concatenarea şirurilor

```
String s1="Ana";
String s2="mere";
int x=10;
System.out.println(s1 + " are " + x + " " + s2);
```

- operatori pentru conversii (cast) : (tip-de-data)

```
int a = (int)'a';
char c = (char)96;
int i = 200;
long l = (long)i; //widening conversion
long l2 = (long)200;
int i2 = (int)l2; //narrowing conversion
```

Comentarii

Există trei feluri de comentarii:

- Comentarii pe mai multe linii, închise între /* şi */.
- Comentarii pe mai multe linii care ţin de documentaţie, închise între /** şi */.
  - Textul dintre cele două secvenţe este automat mutat în documentaţia aplicaţiei de către generatorul automat de documentaţie javadoc.
- Comentarii pe o singură linie, care incep cu //.

Tipuri de date

Tipurile primitive:

- aritmetice
  - întregi: byte (1 octet), short(2), int (4), long (8)
  - reale: float (4), double (8)
- caracter: char (2)
- logic: boolean (true şi false)

Tipul referinţă:

- Vectorii, clasele şi interfeţele
- Valoarea unei variabile de acest tip este, spre deosebire de tipurile primitive, o referinţă (adresă de memorie) către valoarea sau mulţimea de valori reprezentată de variabila respectivă.

Nu există: pointer, struct şi union.

Variabile

- Declararea variabilelor:

  Tip numeVariabila;

- Iniţializarea variabilelor:

  Tip numeVariabila = valoare;

- Declararea constantelor:

  final Tip numeVariabila;

  final double PI = 3.14;

  int valoare = 100;

  long numarElemente = 12345678L;

  String bauturaMeaPreferata = "apa";

Categorii variabile

a. Variabile membre ale unei clase, declarate în interiorul unei clase, vizibile pentru toate metodele clasei respective cât şi pentru alte clase în funcţie de nivelul lor de acces

b. Parametrii metodelor, vizibili doar în metoda respectivă

c. Variabile locale, declarate într-o metodă, vizibile doar în metoda respectivă

d. Variabile locale, declarate într-un bloc, vizibile doar în blocul respectiv.

e. Parametrii de la tratarea excepţiilor

Categorii variabile

```
class Exemplu {
 int a;
 public void metoda(int b) {
    a = b;
    int c = 10;
    for(int d=0; d < 10; d++) {
          c --;
    }
    try {
          a = b/c;
    } catch(ArithmeticException e) {
        System.err.println(e.getMessage());
    }
  }
}
```

Instrucţiuni

- Instrucţiuni de decizie:

  if-else, switch-case

- Instrucţiuni de salt:

  for, while, do-while

- Instrucţiuni pentru tratarea excepţiilor:

  try-catch-finally, throw

- Alte instrucţiuni:

  break, continue, return, label:

Instrucţiuni de decizie

- **if-else**

```
if (expresie-logica) {
        ...
}
```

Sau:

```
if (expresie-logica) {
        ...
} else {
        ...
}
```

- **switch-case**

```
switch (variabila) {
case valoare1:
        ...
        break;
case valoare2:
        ...
        break;
default:
        ...
}
```

Instrucţiuni de salt

- **for**

```
for(initializare; expresie-logica; pas-iteratie) {
        //Corpul buclei
}
```

Exemplu:

```
for(int i=0, j=100 ; i < 100 && j > 0; i++, j--) {
    ...
}
```

- **while**

```
while (expresie-logica) {
    ...
}
```

- **do-while**

```
do {
    ...
} while (expresie-logica);
```

Alte instrucţiuni

- break: părăseşte forţat corpul unei structuri repetitive.
- continue: termină forţat iteraţia curentă a unui ciclu şi trece la următoarea iteraţie.

- return [valoare]: termină o metodă şi, eventual, returnează o valoare.

- numeEticheta: defineşte o etichetă.
- Nu există goto
- Pot fi definite etichete folosite astfel:
  - break numeEticheata
  - continue numeEticheta

Exemplu de folosire a etichetelor

```
i=0;
eticheta:
while (i < 10) {
    System.out.println("i="+i);
    j=0;
    while (j < 10) {
        j++;
        if (j==5) continue eticheta;
        //sau: if (j==5) break eticheta;
        System.out.println("j="+j);
    }
    i++;
}
```

Vectori

- Declararea

  Tip[] numeVector; sau

  Tip numeVector[];

- Instanţierea

  numeVector = new Tip[nrElemente];

  v = new int[10];

// alocare spatiu pentru 10 intregi: 40 octeti

  c = new char[10];

//aloca spatiu pentru 10 caractere: 20 octeti

- Iniţializarea (opţional)

  String culori[] ={"Rosu", "Galben"};

Vectori

- Declararea şi instanţierea pot fi făcute simultan:

Tip[] numeVector = new Tip[nrElemente];

- Primul indice al unui vector este 0, deci poziţiile unui vector cu n elemente vor fi cuprinse între 0 şi n - 1.
- Nu sunt permise construcţii de genul *Tip numeVector[nrElemente]*, alocarea memoriei făcându-se doar prin intermediul operatorului new.

int v[10]; //ilegal

int v[] = new int[10]; //corect

v[10]= 3; - exceptie ArrayIndexOutOfBoundsException

Tablouri multidimensionale

Tablou multidimensional = vector de vectori.

Tip mat[][] = new Tip[nrLinii][nrColoane];

- *mat[i]* este linia i a matricii şi reprezintă un vector cu nrColoane elemente iar *mat[i][j]* este elementul de pe linia i şi coloana j.

- Dimensiunea unui vector

  Variabila length:

  int []a = new int[5];

  // a.length are valoarea 5

  int m[][] = new int[5][10];

  // m[0].length are valoarea 10

Copierea vectorilor

```
int a[] = {1, 2, 3, 4};
int b[] = new int[4];

// Varianta 1
for(int i=0; i<a.length; i++)
    b[i] = a[i];

// Varianta 2
System.arrayCopy(a, 0, b, 0, a.length);

// Nu are efectul dorit
    b = a;
```

Sortarea vectorilor - clasa Arrays

Metode din java.util.Arrays:

//import java.util.Arrays; => Arrays.sort(v);

- sort - (QuickSort -O(n log(n)))

```
int v[]={3, 1, 4, 2};
java.util.Arrays.sort(v);
// Sorteaza vectorul v
// Acesta va deveni {1, 2, 3, 4}
```

- binarySearch
- equals
- fill

Vectori cu dimensiune variabilă şi eterogeni:

- Vector, ArrayList, etc. din pachetul java.util.

Argumente de la linia de comandă

- Trimiterea argumentelor

```
java NumeAplicatie [arg0 arg1 . . .]
java Test Java "Hello Duke" 1.5
```

- Primirea argumentelor:

```
public static void main (String args[]){
   /* args[0] va fi “Java“
   args[1] va fi "Hello Duke“
   s.a.m.d.
   */
}
```

- Numărul argumentelor:

```
public static void main (String args[]) {
   int numarArgumente = args.length ;
}
```

Exemplu

```
public class Salut {
  public static void main (String args[]) {
    if (args.length == 0) {
          System.out.println( "Numar insuficient de
          argumente!");
          System.exit(-1); //termina aplicatia
    }
    String nume = args[0]; //exista sigur
    String prenume;
    if (args.length > 1)
          prenume = args[1];
    else
          prenume = ""; //valoare implicita
    System.out.println("Salut " + nume + "      " +
    prenume);
  }
}
```

Afişarea argumentelor

```
public class Afisare {
   public static void main (String[] args) {
      for (int i = 0; i < args.length; i++)
          System.out.println(args[i]);
   }
}
```

java Afisare Hello Java

Hello
Java

java Afisare ”Hello Java”
Hello Java

Argumente numerice

- Argumentele de la linia de comandă sunt siruri de caractere
- Transformarea şir - număr se face cu metode de tipul:
  - Integer.parseInt
  - Double.parseDouble
  - Float.parseFloat, etc.

```
public class Power {
  public static void main(String args[]) {
    double numar = Double.parseDouble(args[0]);
    int putere = Integer.parseInt(args[1]);
    System.out.println("Rezultat=" + Math.pow(numar,
                                          putere));
   }
}
```

Exemplu clasa si apel metoda clasa

```
class Prim
{
  boolean ePrim (int n){
     for ( int i=2; i<=Math.sqrt(n); i++ )
          if (n%i = = 0) return false;
     return true;
  }
  public static void main( String args[])
  {
      int n=1453;
      Prim p=new Prim();
       if( p.ePrim(n) )
            System.out.println(n +“ e prim");
        else
            System.out.println(n+“ nu e prim");
   }
}
```

Şiruri de caractere

- char[]
- String

  String s = "abc";

  String s = new String("abc");

  char data[] = {’a’, ’b’, ’c’};

  String s = new String(data);
- StringBuffer

  Un şir de caractere constant (nu se doresc schimbări în porţiuni din conţinutul său pe parcursul execuţiei programului) va fi declarat de tipul String, altfel va fi declarat de tip StringBuffer. StringBuffer pune la dispoziţie metode pentru modificarea conţinutului şirului, cum ar fi: append, insert, delete, reverse.

Şiruri de caractere

- Uzual, cea mai folosită modalitate de a lucra cu şiruri este prin intermediul clasei String.
- Clasa StringBuffer va fi utilizată predominant în aplicaţii dedicate procesării textelor cum ar fi editoarele de texte.

```
String s1="asd",s2="";
s1=s2+"a";
System.out.println(s1+" "+s2);
```

- Testarea egalităţii: equals

  ```
  if (nume.equals("duke")) { ... }
  ```

Şiruri de caractere

- Concatenarea: +

  String s1 = "ab" + "cd";

  String s2 = s1 + 123 + "xyz"
- extrem de flexibil, permite concatenarea şirurilor cu obiecte de orice tip care au o reprezentare de tip şir de caractere.

Exemple:

System.out.print("Vectorul v are" + v.length + "elem.");

String x = "a" + 1 + "b“;

De fapt:

String x = new StringBuffer().append("a").append(1).

append("b").toString()

- Obs: şirul s=1+2+"a"+1+2 va avea valoarea "3a12", primul + fiind operatorul matematic de adunare iar al doilea +, cel de concatenare a şirurilor.